

Datazap Partner API

C# Integration Guide · v1.1

.NET 8 · OAuth 2.0 Authorization Code + PKCE · HttpClient · .NET MAUI and desktop

Introduction

Datazap lets your app upload data logs straight into a user's Datazap account. The user approves your app once on a Datazap consent screen, your app receives an access token, and from then on it can upload CSV logs and list the user's projects on their behalf. This guide is long because it includes a complete, reviewed C# client and handles the edge cases; the integration itself is small.

The process in three parts

1. **Connect once.** Your app opens the Datazap consent page in the system browser. The user signs in and taps *Authorize*. Your app receives an access token and a refresh token and stores them securely. Steps 1 to 6.
2. **Upload logs.** Your app sends one or more CSV files to a single endpoint with that token, optionally into a project the user picked from a list. Steps 7 to 9.
3. **Stay connected.** The access token lasts 24 hours and the client class in this guide refreshes it for you. The user can disconnect your app from their Datazap settings at any time, and your app finds out on its next call. Step 6 and the [Error](#) reference.

Two ways to use it

- **Send to Datazap.** Add a button to a saved log or a multi-select of logs. The user can optionally choose a project and add notes, then taps Send. It's the simplest integration and works well on its own.
- **Auto-upload.** An opt-in setting that queues each completed logging session and uploads it in the background. Once enabled, logs appear in Datazap automatically, with no extra steps from the user. It uses the same API, with additional handling for queueing, retries, connectivity, and plan limits.

Both patterns are covered with code under *Integration patterns* after the shared client implementation. Everything before that applies to both.

How this guide is organized

Section	What it covers
Steps 1 to 6	Connecting: PKCE, the consent screen on mobile and desktop, token exchange, secure storage, and a reusable <code>DatazapClient</code> class that refreshes and retries
Steps 7 to 9	The API: list projects, upload logs, and an optional plan and usage call
Putting it together	A complete connect, list, upload example with error handling
Integration patterns	Send to Datazap and Auto-upload, including a background queue drain
Reference	Error codes, a testing checklist, confidential clients, and what to send us to go live

Overview

Datazap uses OAuth 2.0 Authorization Code with PKCE. The authorization flow follows the standard protocol, with one Datazap-specific difference: the token endpoint accepts a JSON body rather than `application/x-www-form-urlencoded`, so libraries that assume form-encoded token requests may need a custom token exchange. Everything in this guide is written against plain `HttpClient` and `System.Text.Json`, so it drops into a .NET MAUI, WPF, WinForms, or console project with no extra packages.

Getting set up

You send us the redirect URIs your app will use (deep links or loopback URLs, see Step 3) and we send back a `client_id`, the public identifier for your app. The full checklist is under *Going live*.

There is no client secret. Your app is registered as a *public client*: a mobile or desktop binary cannot keep a secret, so PKCE and the registered redirect URI are what protect the exchange, and the token endpoint takes the `client_id` alone. See *Confidential clients* at the end if your integration runs on a server instead.

Quick reference

Item	Value
Consent page (open in the system browser)	<code>https://datazap.me/integrations/authorize</code>
Token endpoint (JSON body)	<code>POST https://datazap.me/api/integrations/token</code>
Revoke endpoint (JSON body)	<code>POST https://datazap.me/api/integrations/revoke</code>
List projects	<code>GET https://datazap.me/api/v1/projects</code>
Upload logs (multipart)	<code>POST https://datazap.me/api/v1/logs/upload</code>

Plan and usage (optional)	GET <code>https://datazap.me/api/v1/me</code>
Access token lifetime	24 hours (<code>expires_in</code> is in seconds)
Refresh token	Rotates on every refresh. Stays valid until the user disconnects or re-authorizes.
Client type	Public: PKCE only, no client secret
PKCE	Required, <code>S256</code> only
Redirect URI matching	Exact, except loopback <code>http://127.0.0.1</code> URIs match on any port
Rate limit	60 requests per minute per access token on <code>/api/v1/*</code>
Max files per upload request	Free 3, Pro 10, Tuner 20, by the user's plan (see Step 8). <code>.csv</code> only.
Error responses	JSON with a readable <code>error</code> and a stable <code>code</code> (see Error reference)

Two things that trip up OAuth libraries

The token endpoint takes a JSON body, not `application/x-www-form-urlencoded`. And a custom-scheme `redirect_uri` must match a registered URI character for character; only loopback URIs get a free port.

How the flow fits together

1. Generate a PKCE verifier and challenge plus a random `state`.
2. Open the consent page in the system browser with your `client_id`, redirect URI, scopes and challenge.
3. The user signs in if needed and taps *Authorize*. Datazap redirects to your redirect URI with `code` and `state`.
4. Exchange the code (plus the PKCE verifier) for an access token and a refresh token.
5. Call the API with `Authorization: Bearer <access_token>`. Refresh when you get a 401.

Step 1 • PKCE helper

The verifier is 32 random bytes as lowercase hex (64 characters, inside the 43 to 128 range PKCE allows). The challenge is the SHA-256 of the verifier's ASCII bytes, base64url encoded without padding. Datazap hashes the verifier string exactly as sent, so keep it as plain ASCII.

```
Pkce.cs

using System.Security.Cryptography;
using System.Text;

public static class Pkce
{
    public static (string Verifier, string Challenge) Create()
    {
        var bytes = RandomNumberGenerator.GetBytes(32);
        var verifier = Convert.ToHexString(bytes).ToLowerInvariant();
        var challenge = Base64Url(SHA256.HashData(Encoding.ASCII.GetBytes(verifier)));
        return (verifier, challenge);
    }

    // Echoed back in the redirect; compare it to stop CSRF
    public static string RandomState() => Base64Url(RandomNumberGenerator.GetBytes(16));

    public static string Base64Url(byte[] bytes) =>
        Convert.ToBase64String(bytes).TrimEnd('=').Replace('+', '-').Replace('/', '_');
}

```

Step 2 · Build the authorization URL

`scope` is the permission list the consent screen shows the user. Send both, space separated: `logs:write` to upload logs and `projects:read` to list projects so the user can pick where uploads go.

`Uri.EscapeDataString` handles the encoding.

```
public static Uri BuildAuthorizeUrl(
    string clientId, string redirectUri, string codeChallenge, string state)
{
    var query = new Dictionary<string, string>
    {
        ["response_type"] = "code",
        ["client_id"] = clientId,
        ["redirect_uri"] = redirectUri,
        ["scope"] = "logs:write projects:read",
        ["code_challenge"] = codeChallenge,
        ["code_challenge_method"] = "S256",
        ["state"] = state,
    };
    var qs = string.Join("&", query.Select(kv => $"{Enc(kv.Key)}={Enc(kv.Value)}"));
    return new Uri($"https://datazap.me/integrations/authorize?{qs}");

    static string Enc(string value) => Uri.EscapeDataString(value);
}

```

If the user is not signed in to Datazap they are sent to the login page and returned to the consent screen afterwards. The consent screen shows your app name, logo, description and the requested permissions.

Step 3 · Open the browser and receive the callback

Always use the system browser (or the platform's auth session), never an embedded WebView. Pick the option that matches your app.

For the redirect URI on mobile, use a custom scheme in reverse-domain form that only you would claim, for example `com.yourcompany.yourapp://datazap/callback`, rather than a short generic scheme another app could register too. A claimed HTTPS link (Android App Links, iOS Universal Links) is stronger still if you already have one; register it with us as-is.

Option A · .NET MAUI (Android / iOS)

`WebAuthenticator` opens a Custom Tab on Android and an `ASWebAuthenticationSession` on iOS, then hands you the query string of the redirect. Register the scheme once per platform.

```
Platforms/Android/WebAuthenticationCallbackActivity.cs

using Android.App;
using Android.Content;
using Android.Content.PM;
using Microsoft.Maui.Authentication;

[Activity(NoHistory = true, LaunchMode = LaunchMode.SingleTop, Exported = true)]
[IntentFilter(new[] { Intent.ActionView },
    Categories = new[] { Intent.CategoryDefault, Intent.CategoryBrowsable },
    DataScheme = "com.yourcompany.yourapp", DataHost = "datazap")]
public class WebAuthenticationCallbackActivity : WebAuthenticatorCallbackActivity
{
}
```

```
Platforms/Android/AndroidManifest.xml (inside <manifest>; Android 11+ needs this to see Custom Tabs)

<queries>
  <intent>
    <action android:name="android.support.customtabs.action.CustomTabsService" />
  </intent>
</queries>
```

```
Platforms/iOS/Info.plist (inside the top-level <dict>)

<key>CFBundleURLTypes</key>
<array>
  <dict>
    <key>CFBundleURLName</key>
    <string>Datazap callback</string>
    <key>CFBundleURLSchemes</key>
    <array><string>com.yourcompany.yourapp</string></array>
  </dict>
</array>
```

```

public async Task<string> RunConsentAsync(
    Uri authorizeUrl, string redirectUri, string state, CancellationToken ct)
{
    WebAuthenticatorResult result;
    try
    {
        // WaitAsync lets the caller's token bound the browser round trip
        result = await WebAuthenticator.Default
            .AuthenticateAsync(authorizeUrl, new Uri(redirectUri))
            .WaitAsync(ct);
    }
    catch (OperationCanceledException)
    {
        // The user closed the browser, or the caller's token fired
        throw new DatazapAuthException("The authorization flow was cancelled.");
    }

    // Check state first: an unsolicited callback must not even produce a "declined" message
    if (result.Get("state") != state)
        throw new DatazapAuthException("State mismatch. Discard this response.");

    // The user tapped Deny: the redirect carries error=access_denied instead of a code
    if (result.Properties.TryGetValue("error", out var error))
        throw new DatazapAuthException(error == "access_denied"
            ? "The user declined."
            : result.Get("error_description") ?? error);

    return result.Get("code")
        ?? throw new DatazapAuthException("No authorization code in the callback.");
}

```

Option B · Windows / macOS desktop (loopback redirect)

Desktop apps listen on a loopback port and open the default browser. Register

`http://127.0.0.1/callback` with us once, without a port: loopback redirect URIs match on any port, so the app can bind whichever port is free at runtime and put that port in the authorization request. Because the port changes per attempt, the redirect URI is built inside `ConnectAsync` (Step 6) rather than fixed in the constructor: pass `null` as the redirect URI and the client takes this path.

```

using System.Diagnostics;
using System.Net;
using System.Net.Sockets;
using System.Text;
using System.Web;

// Asks the OS for a free port and releases it. HttpListener cannot bind port 0 itself,
// so another process could grab the port in between. Rare enough that the sample does
// not retry; wrap Start() in a small retry loop if you want to close that window.
public static int FreeLoopbackPort()
{
    var probe = new TcpListener(IPAddress.Loopback, 0);

```

```

probe.Start();
var port = ((IPEndPoint)probe.LocalEndpoint).Port;
probe.Stop();
return port;
}

// redirectUri is the exact "http://127.0.0.1:{port}/callback" from the authorize URL
public static async Task<string> RunConsentOnLoopbackAsync(
    Uri authorizeUrl, string redirectUri, string expectedState, CancellationToken ct)
{
    var port = new Uri(redirectUri).Port;
    using var listener = new HttpListener();
    listener.Prefixes.Add($"http://127.0.0.1:{port}/"); // root prefix, path checked below
    listener.Start();

    Process.Start(new ProcessStartInfo(authorizeUrl.ToString()) { UseShellExecute = true });

    while (true)
    {
        var context = await listener.GetContextAsync().WaitAsync(ct); // ct bounds the wait
        if (context.Request.Url?.AbsolutePath != "/callback")
        {
            context.Response.StatusCode = 404;
            context.Response.Close();
            continue;
        }

        var query = HttpUtility.ParseQueryString(context.Request.Url.Query);
        var html = "<html><body>Done. You can return to the app.</body></html>";
        var bytes = Encoding.UTF8.GetBytes(html);
        context.Response.ContentType = "text/html";
        await context.Response.OutputStream.WriteAsync(bytes);
        context.Response.Close();

        if (query["state"] != expectedState)
            throw new DatazapAuthException("State mismatch. Discard this response.");
        if (query["error"] is { } error)
            throw new DatazapAuthException(error == "access_denied"
                ? "The user declined."
                : query["error_description"] ?? error);
        return query["code"]
            ?? throw new DatazapAuthException("No authorization code in the callback.");
    }
}

```

Step 4 · Exchange the code for tokens

One JSON POST with no client secret. The response carries both tokens; the access token expires in 24 hours. Errors come back as `{ "error": "invalid_grant", "error_description": "..."} with a 400.`

```

using System.Net.Http.Json;
using System.Text.Json.Serialization;

public sealed class TokenResponse
{
    [JsonPropertyName("access_token")] public string AccessToken { get; set; } = "";
    [JsonPropertyName("refresh_token")] public string RefreshToken { get; set; } = "";
    [JsonPropertyName("token_type")] public string TokenType { get; set; } = "Bearer";
    [JsonPropertyName("expires_in")] public int ExpiresIn { get; set; }
    [JsonPropertyName("scope")] public string Scope { get; set; } = "";
}

public sealed class OAuthError
{
    [JsonPropertyName("error")] public string Error { get; set; } = "";
    [JsonPropertyName("error_description")] public string? Description { get; set; }
}

var tokenUrl = "https://datazap.me/api/integrations/token";
using var response = await http.PostAsJsonAsync(tokenUrl, new
{
    grant_type = "authorization_code",
    code, // from the callback
    client_id = clientId,
    code_verifier = verifier, // from Pkce.Create(), same run as the challenge
    redirect_uri = redirectUri, // identical to the one in the authorize URL
});

if (!response.IsSuccessStatusCode)
{
    var err = await response.Content.ReadFromJsonAsync<OAuthError>();
    throw new DatazapAuthException($"{err?.Error}: {err?.Description}");
}

var tokens = await response.Content.ReadFromJsonAsync<TokenResponse>();

```

Step 5 · Store the tokens

Persist the access token, the refresh token and the computed expiry. On MAUI use `SecureStorage` (Keychain on iOS, Keystore on Android). On Windows use `ProtectedData` or the credential manager. Never log tokens.

```

public sealed record StoredTokens(
    string AccessToken, string RefreshToken, DateTimeOffset ExpiresAt);

public interface ITokenStore
{
    Task<StoredTokens?> LoadAsync();
    Task SaveAsync(StoredTokens tokens);
    Task ClearAsync();
}

```

```
// MAUI implementation
public sealed class SecureTokenStore : ITokenStore
{
    private const string Key = "datazap_tokens";

    public async Task<StoredTokens?> LoadAsync()
    {
        var json = await SecureStorage.Default.GetAsync(Key);
        return json is null ? null : JsonSerializer.Deserialize<StoredTokens>(json);
    }

    public Task SaveAsync(StoredTokens tokens) =>
        SecureStorage.Default.SetAsync(Key, JsonSerializer.Serialize(tokens));

    public Task ClearAsync()
    {
        SecureStorage.Default.Remove(Key);
        return Task.CompletedTask;
    }
}
```

Step 6 · A client that refreshes and retries

The class below covers token exchange, refresh, and the Datazap API calls. A few details matter and are easy to get wrong, so they are built in:

- **Refresh tokens rotate.** Every refresh returns a new refresh token and invalidates the old one. Two refreshes racing each other leave you with a dead session, so refreshes are serialized with a `SemaphoreSlim` and re-checked after the wait. Keep one `DatazapClient` per app; the lock only coordinates calls through the same instance.
- **Persist the refresh response before anything else.** If the network drops after Datazap rotated the tokens but before your app stored the new pair, that installation has to reconnect. The client saves first and returns second for that reason.
- **Only `invalid_grant` ends the session.** A refresh that fails with a 5xx or a 429 is a temporary problem and keeps the stored tokens. Only `invalid_grant` (the user disconnected, or authorized again elsewhere) clears them.
- **Requests are built by a factory.** An `HttpRequestMessage` can only be sent once, so a retry after a 401 needs a fresh request and freshly opened file streams. Each request is disposed after sending, which closes those streams.
- **No 100-second surprise.** `HttpClient` defaults to a 100-second timeout, which a batch of large logs on cellular can exceed. The client disables it and every call takes a `CancellationToken` so you decide the bound.

```
DatazapClient.cs
using System.Net;
using System.Net.Http.Headers;
```

```

using System.Net.Http.Json;
using System.Text.Json;
using System.Text.Json.Serialization;

public sealed record Project(
    [property: JsonPropertyName("id")] string Id,
    [property: JsonPropertyName("name")] string Name);

public sealed record UploadedLog(
    [property: JsonPropertyName("id")] string Id,
    [property: JsonPropertyName("filename")] string Filename,
    [property: JsonPropertyName("duplicate")] bool Duplicate);

public sealed record Account(
    [property: JsonPropertyName("username")] string Username,
    [property: JsonPropertyName("plan")] string Plan,
    [property: JsonPropertyName("planName")] string PlanName,
    [property: JsonPropertyName("limits")] AccountLimits Limits,
    [property: JsonPropertyName("usage")] AccountUsage Usage);

public sealed record AccountLimits(
    [property: JsonPropertyName("maxLogs")] int? MaxLogs,
    [property: JsonPropertyName("maxFileSizeMb")] int MaxFileSizeMb,
    [property: JsonPropertyName("maxFilesPerUpload")] int MaxFilesPerUpload);

public sealed record AccountUsage(
    [property: JsonPropertyName("logs")] int Logs);

// OpenRead is a factory so a retried request can re-open the file.
// ExternalId (your own id, e.g. a session id) makes a retried upload return the existing log.
public sealed record LogFile(
    string FileName, Func<Stream> OpenRead, string? Note = null, string? ExternalId = null);

public sealed class DatazapClient
{
    // No global timeout: callers bound each call with their CancellationToken
    private readonly HttpClient _http = new()
    {
        BaseAddress = new Uri("https://datazap.me/"),
        Timeout = Timeout.InfiniteTimeSpan,
    };
    private readonly ITokenStore _store;
    private readonly SemaphoreSlim _refreshLock = new(1, 1);

    public string ClientId { get; }
    // Custom scheme on mobile; null on desktop, where a loopback URI is built per attempt
    public string? RedirectUri { get; }

    public DatazapClient(string clientId, string? redirectUri, ITokenStore store)
    {
        ClientId = clientId;
        RedirectUri = redirectUri;
        _store = store;
    }
}

```

```

public async Task<bool> IsConnectedAsync() => await _store.LoadAsync() is not null;

// ---- Connect / disconnect (Steps 1 to 5) ----

public async Task ConnectAsync(CancellationTokens ct = default)
{
    var (verifier, challenge) = Pkce.Create();
    var state = Pkce.RandomState();

    // Mobile uses the fixed scheme; desktop builds a loopback URI with a port picked now.
    // The same string must go into the authorize URL and the token exchange.
    var redirectUri = RedirectUri ?? $"http://127.0.0.1:{FreeLoopbackPort()}/callback";
    var authorizeUrl = BuildAuthorizeUrl(ClientId, redirectUri, challenge, state);

    var code = RedirectUri is not null
        ? await RunConsentAsync(authorizeUrl, redirectUri, state, ct)
        : await RunConsentOnLoopbackAsync(authorizeUrl, redirectUri, state, ct);

    using var response = await _http.PostAsJsonAsync("api/integrations/token", new
    {
        grant_type = "authorization_code",
        code,
        client_id = ClientId,
        code_verifier = verifier,
        redirect_uri = redirectUri,
    }, ct);
    await _store.SaveAsync(await ReadTokensAsync(response, ct));
}

public async Task DisconnectAsync(CancellationTokens ct = default)
{
    // Serialize token mutations so a queued refresh cannot restore a disconnected session
    await _refreshLock.WaitAsync(ct);
    try
    {
        var tokens = await _store.LoadAsync();
        if (tokens is null) return;
        try
        {
            // Best effort: revoke server-side so it disappears from the user's settings
            using var response = await _http.PostAsJsonAsync(
                "api/integrations/revoke",
                new { token = tokens.RefreshToken, client_id = ClientId },
                ct);
        }
        finally
        {
            await _store.ClearAsync(); // local disconnect succeeds even if revoke fails
        }
    }
    finally
    {
        _refreshLock.Release();
    }
}

```

```

    }
}

// ---- API calls (Steps 7 to 9) ----

public async Task<IReadOnlyList<Project>> GetProjectsAsync(CancellationToken ct = default)
{
    using var response = await SendAsync(() =>
        new HttpRequestMessage(HttpMethod.Get, "api/v1/projects"), ct);
    await EnsureSuccessAsync(response, ct);
    return (await response.Content.ReadFromJsonAsync<ProjectsResponse>(ct))!.Projects;
}

public async Task<IReadOnlyList<UploadedLog>> UploadLogsAsync(
    IReadOnlyList<LogFile> files, string? projectId = null, CancellationToken ct =
default)
{
    // 20 is the API-wide ceiling; the plan may allow fewer (403 max_files_per_upload)
    if (files.Count is 0 or > 20)
        throw new ArgumentException("Send between 1 and 20 files per request.");

    using var response = await SendAsync(() =>
    {
        var form = new MultipartFormDataContent();
        foreach (var file in files)
        {
            var part = new StreamContent(file.OpenRead());
            part.Headers.ContentType = new MediaTypeHeaderValue("text/csv");
            // Field name "files"; the file name must end in .csv
            form.Add(part, "files", file.FileName);
        }
        if (projectId is not null)
            form.Add(new StringContent(projectId), "projectId");
        if (files.Any(f => f.Note is not null))
        {
            var notes = JsonSerializer.Serialize(files.Select(f => f.Note).ToArray());
            form.Add(new StringContent(notes), "notes");
        }
        if (files.Any(f => f.ExternalId is not null))
        {
            var ids = JsonSerializer.Serialize(files.Select(f => f.ExternalId).ToArray());
            form.Add(new StringContent(ids), "externalIds");
        }
        return new HttpRequestMessage(HttpMethod.Post, "api/v1/logs/upload")
        {
            Content = form,
        };
    }, ct);

    await EnsureSuccessAsync(response, ct);
    return (await response.Content.ReadFromJsonAsync<UploadResponse>(ct))!.Logs;
}

// Optional (Step 9): plan, limits and usage for the connected account

```

```

public async Task<Account> GetAccountAsync(CancellationTokent ct = default)
{
    using var response = await SendAsync(() =>
        new HttpRequestMessage(HttpMethod.Get, "api/v1/me"), ct);
    await EnsureSuccessAsync(response, ct);
    return (await response.Content.ReadFromJsonAsync<Account>(ct))!;
}

// ---- Token plumbing (Step 6) ----

private async Task<HttpResponseMessage> SendAsync(
    Func<HttpRequestMessage> build, CancellationTokent ct)
{
    var tokens = await _store.LoadAsync()
        ?? throw new DatazapAuthException(
            "Not connected. Call ConnectAsync first.", null, 401);

    // Refresh a little early so a long upload never straddles the expiry
    if (tokens.ExpiresAt - DateTimeOffset.UtcNow < TimeSpan.FromMinutes(5))
        tokens = await RefreshAsync(tokens, ct);

    var response = await SendWithTokenAsync(build, tokens.AccessToken, ct);
    if (response.StatusCode != HttpStatusCode.Unauthorized) return response;

    // Rejected anyway (clock skew, revoked elsewhere): refresh once and retry once
    response.Dispose();
    tokens = await RefreshAsync(tokens, ct);
    response = await SendWithTokenAsync(build, tokens.AccessToken, ct);

    // Still 401 with a fresh token: the session is gone, make the caller reconnect
    if (response.StatusCode == HttpStatusCode.Unauthorized)
        await _store.ClearAsync();
    return response;
}

private async Task<HttpResponseMessage> SendWithTokenAsync(
    Func<HttpRequestMessage> build, string accessToken, CancellationTokent ct)
{
    using var request = build(); // disposing the request closes the file streams
    request.Headers.Authorization = new AuthenticationHeaderValue("Bearer", accessToken);
    return await _http.SendAsync(request, ct);
}

private async Task<StoredTokens> RefreshAsync(StoredTokens current, CancellationTokent ct)
{
    await _refreshLock.WaitAsync(ct);
    try
    {
        // Re-read under the lock. A disconnect that ran while we waited is final, even if
        // its revoke never reached the server; a refresh that ran is the live token pair.
        var latest = await _store.LoadAsync()
            ?? throw new DatazapAuthException(
                "Not connected. Call ConnectAsync first.", null, 401);
        if (latest.RefreshToken != current.RefreshToken)
    }

```

```

        return latest;

    using var response = await _http.PostAsJsonAsync("api/integrations/token", new
    {
        grant_type = "refresh_token",
        refresh_token = current.RefreshToken,
        client_id = ClientId,
    }, ct);

    if (!response.IsSuccessStatusCode)
    {
        // A proxy can answer a 5xx with HTML; that must still reach the retry path
        OAuthError? err = null;
        try { err = await response.Content.ReadFromJsonAsync<OAuthError>(ct); }
        catch (JsonException) { }

        // Only invalid_grant ends the session (disconnected or re-authorized)
        if (err?.Error == "invalid_grant")
        {
            await _store.ClearAsync();
            throw new DatazapAuthException(
                $"Reconnect required ({err.Error}: {err.Description})",
                err.Error, (int)response.StatusCode);
        }
        if (response.StatusCode == HttpStatusCode.TooManyRequests)
            throw new DatazapRateLimitException(
                err?.Description ?? "Rate limited", RetryAfter(response));

        // 5xx, network hiccup at the edge, etc.: keep the stored tokens and try later
        throw new DatazapApiException(
            (int)response.StatusCode,
            err?.Description ?? err?.Error ?? response.ReasonPhrase
                ?? "Refresh failed",
            err?.Error);
    }

    var stored = await ReadTokensAsync(response, ct);
    await _store.SaveAsync(stored);
    return stored;
}
finally
{
    _refreshLock.Release();
}

private static async Task<StoredTokens> ReadTokensAsync(
    HttpResponseMessage response, CancellationToken ct)
{
    if (!response.IsSuccessStatusCode)
    {
        OAuthError? err = null;
        try { err = await response.Content.ReadFromJsonAsync<OAuthError>(ct); }
        catch (JsonException) { }
    }
}

```

```

        throw new DatazapAuthException(
            $"{err?.Error ?? response.ReasonPhrase}: {err?.Description}",
            err?.Error, (int)response.StatusCode);
    }

    var t = (await response.Content.ReadFromJsonAsync<TokenResponse>(ct))!;
    var expiresAt = DateTimeOffset.UtcNow.AddSeconds(t.ExpiresIn);
    return new StoredTokens(t.AccessToken, t.RefreshToken, expiresAt);
}

private static async Task EnsureSuccessAsync(
    HttpResponseMessage response, CancellationToken ct)
{
    if (response.IsSuccessStatusCode) return;

    ApiError? body = null;
    try { body = await response.Content.ReadFromJsonAsync<ApiError>(ct); }
    catch (JsonException) { }
    var message = body?.Error ?? response.ReasonPhrase ?? "Request failed";

    // Branch on the stable code, never on the English message
    throw response.StatusCode switch
    {
        HttpStatusCode.Unauthorized => new DatazapAuthException(message, body?.Code, 401),
        HttpStatusCode.Forbidden or HttpStatusCode.RequestEntityTooLarge =>
            new DatazapLimitException(
                (int)response.StatusCode, message, body?.Code, body?.Filename),
        HttpStatusCode.TooManyRequests =>
            new DatazapRateLimitException(message, RetryAfter(response)),
        _ => new DatazapApiException((int)response.StatusCode, message, body?.Code),
    };
}

// Datazap sends Retry-After in seconds; the date form is handled for completeness
private static TimeSpan? RetryAfter(HttpResponseMessage response)
{
    var header = response.Headers.RetryAfter;
    if (header?.Delta is { } delta) return delta;
    if (header?.Date is { } date) return date - DateTimeOffset.UtcNow;
    return null;
}

private sealed record ProjectsResponse(
    [property: JsonPropertyName("projects")] List<Project> Projects);
private sealed record UploadResponse(
    [property: JsonPropertyName("logs")] List<UploadedLog> Logs);
private sealed record ApiError(
    [property: JsonPropertyName("error")] string Error,
    [property: JsonPropertyName("code")] string? Code,
    [property: JsonPropertyName("filename")] string? Filename);
}

public class DatazapApiException : Exception
{
    public int Status { get; }
}

```

```

    public string? Code { get; }
    public DatazapApiException(int status, string message, string? code = null) :
base(message)
    {
        Status = status;
        Code = code;
    }
}

// Stored tokens are gone (or never existed): show the connect button.
// Status is the real HTTP status: 401 from the API, 400 from the token endpoint,
// 0 for consent-flow errors that never reached the server.
public sealed class DatazapAuthException : DatazapApiException
{
    public DatazapAuthException(string message, string? code = null, int status = 0)
        : base(status, message, code) { }
}

// 403 or 413: a plan limit, a missing scope, or an inactive app. Code says which.
public sealed class DatazapLimitException : DatazapApiException
{
    public string? Filename { get; }
    public DatazapLimitException(int status, string message, string? code, string? filename)
        : base(status, message, code)
    {
        Filename = filename;
    }
}

public sealed class DatazapRateLimitException : DatazapApiException
{
    public TimeSpan? RetryAfter { get; }
    public DatazapRateLimitException(string message, TimeSpan? retryAfter)
        : base(429, message, "rate_limited")
    {
        RetryAfter = retryAfter;
    }
}

```

Step 7 • List projects

Projects are the folders a user organizes logs into. Offer them as the upload destination; uploading without a `projectId` is also fine and puts the log at the top level of the account. Projects come back newest first.

```
GET /api/v1/projects → 200

{
  "projects": [
    { "id": "cm1x9k2...", "name": "My E30 Build" },
    { "id": "cm1x7q8...", "name": "Customer Car" }
  ]
}
```

Step 8 · Upload logs

Multipart form data with these fields:

Field	Value
<code>files</code>	One part per file, repeated. File name must end in <code>.csv</code> , content type <code>text/csv</code> . The maximum number of files per request depends on the user's plan (table below); over the limit the whole request is rejected with a 403 and nothing is stored.
<code>projectId</code>	Optional. A project id from Step 7. Must belong to the user, otherwise 400 <code>invalid_project</code> .
<code>notes</code>	Optional. A JSON array string with one entry per file in the same order, for example <code>["3rd gear pull", null, "4th gear pull"]</code> .
<code>externalIds</code>	Optional. A JSON array string with one entry per file in the same order, holding your own id for that log (a session id, up to 128 characters), or <code>null</code> . Ids are unique per Datzap account and per partner app, so two users can reuse the same id and your ids never collide with another app's. Sending a file again with an id already uploaded returns the existing log untouched, even if the project or note differ, so retries are safe. Make the id stable and unique across every installation of your app: a GUID, or a session id namespaced by device, never a bare per-device counter, or a replacement phone's session 1234 would be treated as the old phone's.

```
POST /api/v1/logs/upload → 200

{
  "success": true,
  "uploadedCount": 2,
  "logs": [
    { "id": "cm2a1b3...", "filename": "2026-09-17_pull_3rd.csv", "duplicate": false },
    { "id": "cm2a1b4...", "filename": "2026-09-17_pull_4th.csv", "duplicate": false }
  ]
}
```

`uploadedCount` counts new logs only. A file whose `externalId` was seen before comes back with `duplicate: true` and the id of the log that already exists.

Limits depend on the user's Datazap plan, and the server enforces them before anything is stored. Surface the server's `error` message to the user; it already names the plan and the limit.

Plan	Max files per upload request	Max file size	Max logs in account
Free	3	3 MB	100
Pro	10	5 MB	1,000
Tuner	20	10 MB	Unlimited

Values at the time of writing. 20 files per request is the API-wide ceiling. Rely on the 403 / 413 responses, or on `/me`, rather than hard-coding the rest.

Step 9 · Plan and usage (optional)

Everything above works without this call: the upload endpoint rejects anything over the user's plan with a readable message and a code. Use it when you want to show the connected account, size batches to the plan, check file sizes before uploading, or warn before the log cap instead of after. Any valid token can call it, no extra scope. Cache the result; refresh it after a plan-related 403 or once a day, not on every upload.

```
GET /api/v1/me → 200

{
  "username": "jpsimon",
  "plan": "pro",
  "planName": "Pro",
  "scopes": ["logs:write", "projects:read"],
  "limits": { "maxLogs": 1000, "maxFileSizeMb": 5, "maxFilesPerUpload": 10 },
  "usage": { "logs": 87 }
}
```

`maxLogs` is `null` on plans with no cap. `GetAccountAsync()` in the client above returns this as an `Account` record.

Putting it together

```
// Desktop apps pass null as the redirect URI and get a loopback port per attempt
var datazap = new DatazapClient(
    ClientId, "com.yourcompany.yourapp://datazap/callback", new SecureTokenStore());

// Bound the whole operation; uploads on cellular can be slow
using var cts = new CancellationTokSource(TimeSpan.FromMinutes(10));

try
{
    if (!await datazap.IsConnectedAsync())
        await datazap.ConnectAsync(cts.Token); // opens the browser, returns on approval
}
```

```

var projects = await datazap.GetProjectsAsync(cts.Token);
var target = projects.FirstOrDefault(p => p.Name == "My E30 Build").Id;

var uploaded = await datazap.UploadLogsAsync(new[]
{
    new LogFile("2026-09-17_pull_3rd.csv", () => File.OpenRead(path3rd), "3rd gear pull"),
    new LogFile("2026-09-17_pull_4th.csv", () => File.OpenRead(path4th), "4th gear pull"),
}, projectId: target, ct: cts.Token);

// Retried files come back with Duplicate = true and were not created again
var newCount = uploaded.Count(log => !log.Duplicate);
ShowToast(newCount == 0
    ? "These logs are already in Datazap"
    : $"Uploaded {newCount} logs to Datazap");
}
catch (DatazapAuthException)
{
    ShowConnectButton(); // tokens were cleared; the user needs to authorize again
}
catch (DatazapRateLimitException ex)
{
    ScheduleRetry(ex.RetryAfter ?? TimeSpan.FromSeconds(10));
}
catch (DatazapLimitException ex)
{
    ShowMessage(ex.Message); // e.g. "Log limit reached. Your free plan allows 100 logs."
}
catch (DatazapApiException ex)
{
    ShowMessage($"Datazap error {ex.Status}: {ex.Message}");
}
catch (Exception ex) when (ex is HttpRequestException || ex is OperationCanceledException)
{
    ShowMessage("Could not reach Datazap. Check your connection and try again.");
}
}

```

Integration patterns

Send to Datazap

The basic version: a *Send to Datazap* action on a saved log, or on a multi-select of saved logs.

Recommended behavior:

- Send a selection as one request rather than one request per file, up to the plan's `maxFilesPerUpload`. If you would rather not call `/me`, send 3 at a time; every plan allows at least that many.
- Offer the project picker from Step 7 and remember the last choice. Uploading without a project is fine too.
- Offer an optional note field before sending. Notes are the user's to write; whatever they enter is stored per file and shown with the log in Datazap, and leaving it empty is fine.

- Set `ExternalId` to your own id for the log. A second tap on the same log then returns the existing one instead of a copy.

Auto-upload

Once connected, the user toggles *Auto-upload to Datazap* and every completed logging session is queued automatically and uploaded when connectivity and the OS allow. The upload itself is the same call as above; what changes is that it now runs unattended, so the queue must tolerate offline periods, process termination, retries and account limits.

- **Opt in, default off.** Show the toggle only once the account is connected, with a Wi-Fi only sub-option.
- **Persist first, upload second.** When a session ends, durably enqueue the file path, session id, note and project before any network work. The source file must stay on disk until its entry succeeds; if your app cleans up or renames session files, copy pending logs into app-controlled storage.
- **Try right away while the app is alive.** If connectivity suits the user's settings, run one pass immediately. Then hand off to the platform.
- **Android:** enqueue `WorkManager` work with a network constraint (unmetered for Wi-Fi only) and let it own retry and backoff. It survives the app being killed.
- **iOS:** ordinary `HttpClient` calls stop when the app is suspended. For uploads that must finish unattended, use a background `URLSession` upload task, which the system completes in its own process. The request must still be the multipart form from Step 8, and background uploads are file-backed: write the complete multipart body (the `files` parts, `projectId`, `notes`, `externalIds`) to a temporary file, set `Content-Type` with the boundary, and hand that file to the task. Use `BGTaskScheduler` to revisit queued work opportunistically, not as a guarantee. A user force-quit defers uploads until the app is opened again.
- **Retries are safe.** Pass the session id as `ExternalId`. A retry after a timeout gets the same log back with `duplicate: true`. Remove entries from the queue on any successful response, duplicate or not.
- **Branch on error codes, not messages.** `log_limit_reached` pauses the queue. `file_too_large` names the file; park that one and keep the rest. `invalid_project` means the default project was deleted; clear it and retry. Network errors and 5xx back off exponentially; 429 waits for `Retry-After`.
- **Pause, do not flip the toggle.** A full account, a lost authorization, a missing scope or a deactivated app pauses draining and says why. Keep the pause as machine state (the enum below), not as message text, and keep the files queued: these are conditions of the connection, not of any file. Only a problem with a specific request parks files.
- **Clear the pause deliberately.** Nothing clears itself. Set the pause back to `None` after a successful reconnect and whenever the app comes to the foreground; the next pass costs one request and simply re-pauses if the condition still holds, which is also how a freed-up or upgraded account resumes on its own.
- **Default project (nice to have).** A good pattern is a project picker in the auto-upload settings, filled from Step 7, so every auto-upload lands in the project the user chose once. If no project is set, the log goes to the top level of their account, the same as a manual upload without a project. Leave the note empty unless the user set one.

One bounded pass. Background tasks call it with `maxBatches: 1`; the foreground can pass more.

```
public enum AutoUploadPause
{
```

```

    None, ReconnectRequired, LogLimitReached, MissingScope, AppInactive
}

// Set Settings.Pause = AutoUploadPause.None after a reconnect and on app foreground
public async Task DrainQueueAsync(
    DatazapClient datazap, IUploadQueue queue, int maxBatches, CancellationToken ct)
{
    if (!Settings.AutoUploadEnabled || Settings.Pause != AutoUploadPause.None) return;
    if (!await datazap.IsConnectedAsync()) return;

    var onWifi = Connectivity.Current.ConnectionProfiles.Contains(ConnectionProfile.WiFi);
    if (Settings.WifiOnly && !onWifi) return;

    // Cached from /me by foreground code (daily, or when null). Without it: 3 files, 3 MB
    var limits = Settings.CachedLimits ?? new AccountLimits(null, 3, 3);

    for (var pass = 0; pass < maxBatches && !ct.IsCancellationRequested; pass++)
    {
        var batch = await queue.TakeAsync(limits.MaxFilesPerUpload);
        if (batch.Count == 0) return;

        // Missing, unreadable or oversized files never make it into a request; park them
        // one at a time with a reason so one bad entry cannot stop the whole queue
        var maxBytes = limits.MaxFileSizeMb * 1024L * 1024;
        var valid = new List<QueueEntry>();
        foreach (var q in batch)
        {
            try
            {
                var info = new FileInfo(q.Path);
                if (!info.Exists)
                    await queue.ParkAsync(q, "The source log no longer exists.");
                else if (info.Length > maxBytes)
                    await queue.ParkAsync(q, $"Over the {limits.MaxFileSizeMb} MB limit");
                else
                    valid.Add(q);
            }
            catch (Exception ex) when (ex is IOException || ex is UnauthorizedAccessException)
            {
                await queue.ParkAsync(q, ex.Message);
            }
        }
        batch = valid;
        if (batch.Count == 0) continue;

        try
        {
            var files = batch
                .Select(q => new LogFile(
                    q.FileName, () => File.OpenRead(q.Path), q.Note, q.SessionId))
                .ToList();

            await datazap.UploadLogsAsync(files, Settings.DefaultProjectId, ct);
            await queue.RemoveAsync(batch); // a duplicate response is still success
        }
    }
}

```

```

}
catch (DatazapLimitException ex) when (ex.Code == "file_too_large")
{
    // The server named the file: park that one, the rest go back for the next pass
    var culprit = batch.FirstOrDefault(q => q.FileName == ex.Filename);
    if (culprit is not null) await queue.ParkAsync(culprit, ex.Message);
    await queue.RequeueAsync(batch.Where(q => q != culprit));
}
catch (DatazapLimitException ex) when (ex.Code == "max_files_per_upload")
{
    // Plan changed under us. Drop the cache rather than calling /me here: a
    // network call inside a catch would escape the handlers below.
    Settings.CachedLimits = null;
    await queue.RequeueAsync(batch);
    return;
}
catch (DatazapLimitException ex) when (ex.Code == "log_limit_reached")
{
    await PauseAsync(AutoUploadPause.LogLimitReached, ex.Message, batch);
    return;
}
catch (DatazapApiException ex) when (ex.Code is "missing_scope" or "app_inactive")
{
    // Conditions of the connection, not of these files: keep them queued
    var reason = ex.Code == "missing_scope"
        ? AutoUploadPause.MissingScope
        : AutoUploadPause.AppInactive;
    await PauseAsync(reason, ex.Message, batch);
    return;
}
catch (DatazapApiException ex) when (ex.Code == "invalid_project")
{
    Settings.DefaultProjectId = null;    // project was deleted; use top level
    await queue.RequeueAsync(batch);
}
catch (DatazapRateLimitException ex)
{
    await queue.RetryLaterAsync(batch, ex.RetryAfter ?? TimeSpan.FromMinutes(1));
    return;
}
catch (DatazapAuthException)
{
    await PauseAsync(AutoUploadPause.ReconnectRequired,
        "Reconnect Datazap to resume auto-upload.", batch);
    return;
}
catch (Exception ex) when (ex is HttpRequestException
    || ex is OperationCanceledException
    || ex is DatazapApiException { Status: >= 500 })
{
    // Offline, out of background time, or server trouble: exponential backoff
    await queue.RetryLaterAsync(batch, queue.NextBackoff(batch));
    return;
}

```

```

catch (DatazapApiException ex)
{
    // Anything else is a problem with this request itself: park it for the user
    await queue.ParkAsync(batch, ex.Message);
}
catch (Exception ex) when (ex is IOException || ex is UnauthorizedAccessException)
{
    // A file changed after validation. Requeue; the per-file check on the next pass
    // parks the bad entry on its own and the rest go through.
    await queue.RequeueAsync(batch);
    return;
}
}

// The enum is the state the app acts on; the message is only for the notification
async Task PauseAsync(
    AutoUploadPause reason, string message, IEnumerable<QueueEntry> batch)
{
    Settings.Pause = reason;
    NotifyOnce(message);
    await queue.RequeueAsync(batch);
}
}

```

`IUploadQueue`, `QueueEntry`, `Settings` and `NotifyOnce` are placeholders for your own storage, preferences and notification code. `Connectivity` is the .NET MAUI API. On iOS, the upload inside the loop is where you would hand the files to a background `URLSession` instead of awaiting `HttpClient`.

Error reference

API errors are `{ "error": "readable message", "code": "stable_code" }`; 413 also carries `"filename"`. OAuth endpoint errors are `{ "error": "code", "error_description": "message" }` with the standard OAuth codes (`invalid_request`, `invalid_client`, `invalid_grant`, `invalid_scope`, `unsupported_grant_type`). Branch on codes, never on message text. Authorization codes expire after 10 minutes and are single use.

Status	Code	Meaning	What to do
400	<code>invalid_request</code>	Malformed <code>notes</code> or <code>externalIds</code>	Fix the request; not retryable as-is.
400	<code>invalid_project</code>	The <code>projectId</code> does not exist or belongs to someone else	Clear the stored project and retry without it.
400	<code>no_files</code>	No <code>files</code> parts in the request	Fix the request.
401	<code>missing_token</code> , <code>invalid_token</code> , <code>token_expired</code> , <code>token_revoked</code>	The access token is absent, unknown, expired or revoked	The client refreshes once and retries once. If that still fails, tokens are cleared: reconnect.

403	missing_scope	The connection was authorized without this scope	Reconnect requesting the scope.
403	app_inactive	Your app was deactivated on our side	Contact us.
403	max_files_per_upload	More files than the user's plan allows in one request	Split into smaller requests.
403	log_limit_reached	The account is at its plan's log cap	Pause and tell the user once. Nothing to retry until they free space or upgrade.
413	file_too_large	The named filename is over the plan's size cap; nothing was stored	Park that file, resend the others.
429	rate_limited	More than 60 requests in a minute with this access token	Wait for Retry-After seconds, then retry. Batch files into one upload call.
5xx	server_error	Server problem	Retry with backoff. Tell us if it persists.

User experience notes

- Users can disconnect your app at any time under Settings on datazap.me. Your next API call gets a 401, the refresh gets invalid_grant, and the client above clears its tokens.
- One connection per user and app. Authorizing again (for example on a second phone) replaces the earlier connection and its tokens.
- The consent screen shows your app name, logo and description. A square PNG or SVG of at least 128 px looks best.

Testing checklist

1. Connect: the consent screen shows your app name and logo, approving returns to your app with a code, and the exchange succeeds.
2. Deny on the consent screen: your app handles error=access_denied without crashing.
3. List projects, then upload a CSV into one of them and confirm it appears at datazap.me.
4. Upload with an invalid projectId: expect 400 invalid_project with a readable message.
5. Upload the same file twice with the same ExternalId: the second response has uploadedCount: 0 and the same log id with duplicate: true.
6. Force a refresh (set ExpiresAt in the past): the next call succeeds and the stored refresh token changed.

7. Disconnect from Datazap settings, then call the API: the client clears tokens and prompts to reconnect.
8. Turn on airplane mode and upload: the app reports a connection problem rather than crashing, and an auto-upload queue keeps the entry for later.

Confidential clients

If your integration runs on a server you control, for example a web tool or a shop system that syncs from its own backend, we can register it as a *confidential client* instead. It gets a `client_secret`, and the token and revoke calls send it alongside `client_id`; everything else in this guide stays the same. Email **support@datazap.me** if you need that. Apps that ship to users' phones or desktops should stay public.

Going live

Send us:

- App name and a one-line description (shown on the consent screen)
- App logo, square, at least 128 × 128 px, PNG or SVG
- Every redirect URI you will use: custom schemes in reverse-domain form, claimed HTTPS links, and loopback URIs without a port

We register the app and send back your `client_id`. Questions or a stuck integration: **support@datazap.me**.